

Suspension Performance Evaluation

MMS210 Connected fleets in Data-driven Engineering

Group 13

Karl Aadland

Filip Djokic

Jacob Engström

Danilo Rameschandra



June 1, 2026

Contents

1	Introduction	2
1.1	Background	2
1.2	Goal	2
1.3	Limitations	2
2	Method	2
2.1	Testing	2
2.1.1	Steady-State Cornering	2
2.1.2	Longitudinal Acceleration	2
2.1.3	Longitudinal Deceleration	2
3	Engineering pipeline	3
3.1	Extract Raw Data	3
3.2	Signal Processing	5
3.2.1	Tuning	6
3.3	Track Geometry	8
4	Carmaker	10
4.1	Vehicle Parameters	10
4.2	Simulation	11
5	Results & analysis	12
5.1	Cornering	12
5.2	Acceleration & Braking	13
5.3	Normal Force Estimator	13
5.4	Sensitivity Maps	15
5.5	Cost function	16
6	Discussion	17
6.1	Limitations of the Test Cases and scalability	17
6.2	Future work	17

1 Introduction

1.1 Background

The suspension system of a personal vehicle plays a critical role in determining ride comfort, handling performance, stability, and overall vehicle safety. By controlling the interaction between the vehicle body and the road surface, the suspension absorbs shocks, maintains tire contact with the ground, and influences how forces are transmitted through the vehicle during acceleration, braking, and cornering.

1.2 Goal

For this project we want to evaluate the current suspension setups and see how the data can be used to develop a new suspension setup to maximize rider comfort and handling. The setup in mind is a adaptive system that changes the stiffness of the suspension depending on the road conditions and vehicle speeds

1.3 Limitations

Since the logger provided only has an IMU(Inertial Measurement Unit) and a GNSS (Global Navigation Satellite System) we can only measure the vehicles overall response and not directly the suspension. This means that we can get the overall characteristics of how the chassis moves during maneuvers but more detailed suspension data such as displacement, damper velocity and wheel movement relative to the chassis will be missing. Further more the lack of detailed data will make it difficult to understand what part of the suspension contributes to what when it comes to rider comfort and drivability

2 Method

2.1 Testing

The physical testing was conducted at the Stora Holm test area to collect vehicle dynamics data, which serves as the foundation for validating our subsequent Carmaker simulation models. The experiments utilized a Polestar 3 equipped with a provided data acquisition system, consisting of an IMU coupled with a GNSS, to accurately record the vehicle's dynamic responses.

To comprehensively evaluate the vehicle's behavior under different conditions, three specific maneuvers were executed:

2.1.1 Steady-State Cornering

The vehicle was driven along a circular path with a constant radius of 30 meters at a steady speed of 40 km/h.

2.1.2 Longitudinal Acceleration

A straight-line acceleration test was performed from a standstill (0 km/h) up to a target speed of 70 km/h.

2.1.3 Longitudinal Deceleration

A straight-line braking maneuver was executed, bringing the vehicle from 70 km/h down to a complete stop (0 km/h).

The primary objective of this data collection was to establish a robust pipeline for future development and simulation within Carmaker. While physical data was gathered for all three of the vehicle’s pre-configured damping settings across every maneuver, the initial Carmaker vehicle model is developed based on only one specific setting. The data collected from the remaining two damping configurations is reserved to use as a validation dataset, ensuring the accuracy and reliability of the developed simulation pipeline. To ensure reliability and improve the overall robustness of the gathered data, every unique test combination was repeated three times.

3 Engineering pipeline

In this section, the data processing pipeline is described, starting from the extraction of the raw data and ending with the scripts used to compare the simulated with the experimental data.

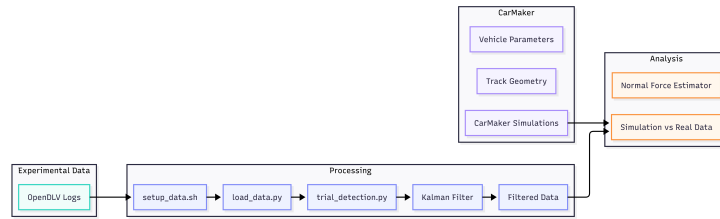


Figure 1: Overview of the Python-based data processing pipeline.

3.1 Extract Raw Data

To streamline the data acquisition process, a Bash script named `setup_data.sh` was developed. The script automatically downloads the selected log files from the OpenDLV server and stores them locally within the project’s `data` directory.

The original log files were downloaded from the OpenDLV platform using `curl`. Since the available download links corresponded to `.log` files, the file suffix was modified to `.csv` using `sed`, enabling the downloaded files to be directly processed in Python.

The downloaded files were stored in:

```
data/raw/opendlv/
```

Moreover, the script automatically creates the required directory structure using `mkdir` and generates a CSV file named `metadata.csv`, which is later used for data organization.

Each log file contains several message types. For this project, only the relevant sensor messages were retained, where the type index identifies either a sensor measurement or a vehicle state. More specifically:

- Type 0: GPS
- Type 1: IMU Accelerometer
- Type 2: IMU Gyroscope

Consequently, the processing pipeline only extracts and processes messages of type 0, type 1, and type 2. Inspection of the accelerometer messages showed that they contain the following variables:

```
1, time, idx, ax, ay, az
```

where **ax**, **ay**, and **az** represent the measured accelerations along the longitudinal, lateral, and vertical axes, respectively.

For the gyroscope, the following variables are available:

```
2, time, idx, wx, wy, wz
```

Lastly, the GPS messages contain:

```
0, time, lat, lon, alt, v_m/s, satellites
```

Based on these message types, a Python script named `load_data.py` was developed to extract the relevant IMU and GPS signals and combine them into a single dataframe named `imu`.

During testing, the logger was not stopped after every individual trial. As a result, each log file contained 3 trials corresponding to the same manoeuvre and suspension setting. Therefore, the complete log files had to be separated into individual CSV files.

This was accomplished using `trial_detection.py`, which contains dedicated detection logic for the two manoeuvre categories:

- `detect_cornering`
- `detect_acc_brake`

The detection is based on GPS speed measurements. For cornering manoeuvres, a trial is detected when the vehicle speed remains above a predefined threshold of 5 km/h for at least 10 s. This criteria prevents the detection of short-duration movements that are not representative of a valid test run.

For acceleration and braking tests, the algorithm identifies the driving segment around the vehicle speed peak and applies a small time padding before and after the manoeuvre to ensure that the complete event is captured.

Figure 2 shows the output of `trial_detection.py` for a cornering manoeuvre. The figure shows the IMU accelerometer and gyroscope signals, the GPS speed profile used for trial segmentation, and the corresponding vehicle trajectory.

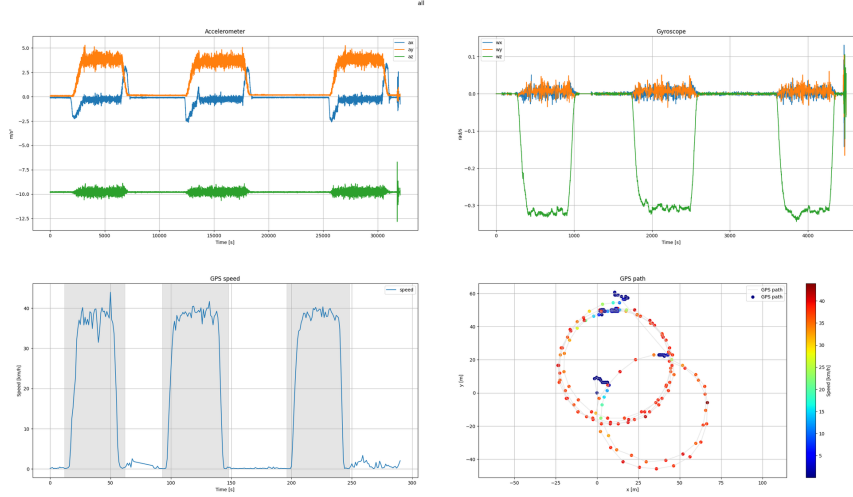


Figure 2: `trial_detection.py` identifying individual cornering trials from a continuous log file.

Apart from the trial detection functions, the `prepare_raw.py` script uses information stored in `metadata.csv` to process each downloaded log file. The metadata file assigns each source log to its corresponding suspension setting and manoeuvre category.

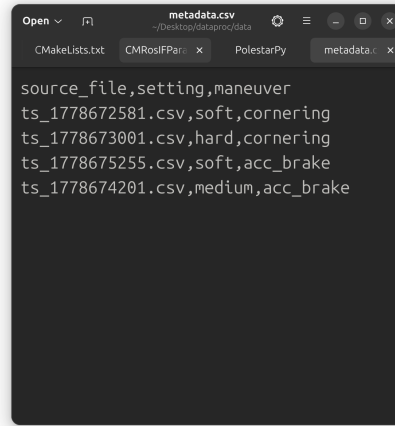


Figure 3: `metadata.csv` file used to associate each log file with a manoeuvre and setting.

Then, for each entry in `metadata.csv`, the `prepare_raw.py` script extracts the IMU and GPS data, detects the individual trials, and stores each trial as a separate CSV file in the corresponding output directory according to the following structure:

```
data/raw/{maneuver}/{setting}/
```

3.2 Signal Processing

After organizing the raw data, a one-dimensional Kalman filter was applied to reduce high-frequency measurement noise while preserving the main trends of the signals. This filtering step improves the quality of the data for subsequent comparison with Carmaker simulation results.

The model assumes that the signal remains approximately constant between consecutive samples, thus any variation in the signal is treated as process uncertainty. This is a reasonable assumption for signal smoothing, as the objective is not to model the complete vehicle dynamics but rather to reduce measurement noise.

$$x_k = x_{k-1} + w_k \quad (1)$$

$$z_k = x_k + v_k \quad (2)$$

Here, x_k denotes the filtered signal, z_k the measured signal, w_k the process noise, and v_k the measurement noise.

For the prediction step, an estimate of the next value without using any measurement:

$$\hat{x}_{k|k-1} = \hat{x}_{k-1} \quad (3)$$

As shown above, the predicted state $\hat{x}_{k|k-1}$ is equal to the previous filtered value, because the model assumes that the signal does not change significantly between two consecutive samples. The covariance P represents the uncertainty of the estimate and during prediction, this uncertainty increases by Q . Similarly, the prediction covariance is computed as:

$$P_{k|k-1} = P_{k-1} + Q \quad (4)$$

For the update step, it uses the prediction step and corrects it using a new measurement z_k :

$$\hat{x}_k = \hat{x}_{k|k-1} + K_k (z_k - \hat{x}_{k|k-1}) \quad (5)$$

Where K_k is the Kalman gain given by:

$$K_k = \frac{P_{k|k-1}}{P_{k|k-1} + R} \quad (6)$$

This gain ultimately decides how much the filter trusts the new measurement compared to the prediction. So, if K_k is close to 1, the filtered signal follows the raw measurement closely. Conversely, if it is close to 0, the filter trusts the previous estimate more and produces a smoother signal.

Since this is a 1D Kalman filter, the filter was applied independently to each accelerometer and gyroscope channel. Once all variables are filtered, a new file is created and saved under:

`data/filtered/{maneuver}/{setting}/`

3.2.1 Tuning

After implementing the filter, different values of Q and R were tested:

Here, Q and R represent the process and measurement noise variances, respectively. In short, these parameters determine the filter tuning.

Setting	Q	R
Strong smooth	0.0001	0.5
Medium smooth	0.001	0.1
Light smooth	0.01	0.05

Table 1: Kalman filter tuning parameters.

If R is large, the filtered signal becomes smoother because the filter trusts the measurement less. Conversely, for a larger Q , the model is trusted less, meaning that the filter becomes more responsive to changes in the measured signal, but also allows more noise to pass through.

Using IMU a_y as an example, the residual:

$$z_k - \hat{x}_k \quad (7)$$

was plotted for the different tuning settings.

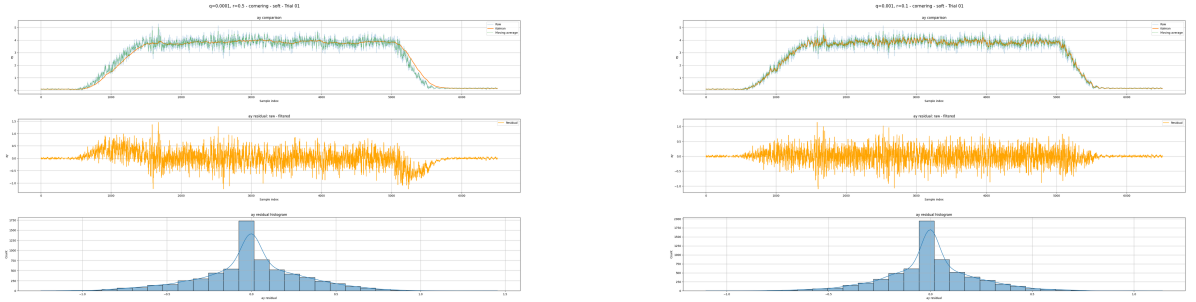


Figure 4: Residual analysis for the strong smooth (left) and medium smooth (right) settings.

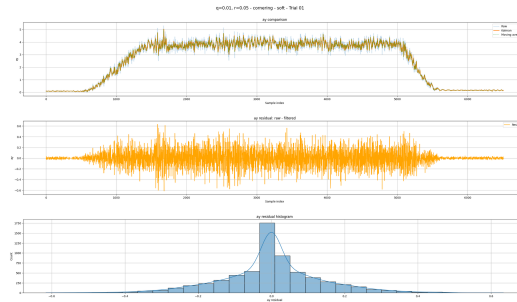


Figure 5: Residual analysis for the light smooth setting.

From the residual plots and histograms, it is possible to observe how aggressive each filter setting is. Therefore, the strong smooth setting was selected, as one of the objectives for this work is to compare real data with the Carmaker one. Since the simulated signals are essentially noise-free, a stronger filtering provides a more appropriate basis for comparison while preserving main trends.

3.3 Track Geometry

Using latitude and longitude coordinates, it is possible to model the track geometry through its dimensions, namely the cornering radius and track length.



Figure 6: Circular track obtained using GPS.

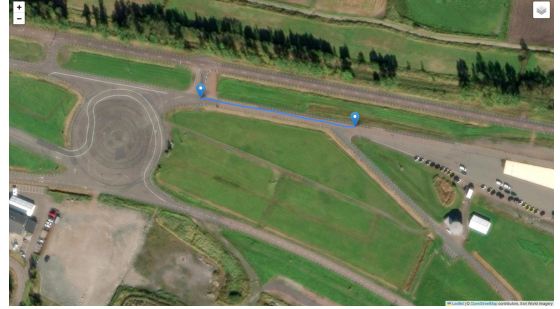


Figure 7: Straight track obtained using GPS.

Within `trial_detection.py`, additional functions were developed to calculate both the cornering radius and the track length using GPS measurements.

For acceleration and braking manoeuvres, the track length is approximated as the sum of the distances between consecutive GPS points:

$$l = \sum_{i=1}^{N-1} \Delta s_i \quad (8)$$

$$\Delta s_i = \sqrt{(x_i - x_{i-1})^2 + (y_i - y_{i-1})^2} \quad (9)$$

For the cornering manoeuvre, the center coordinates of the circle are estimated using the mean of the maximum and minimum coordinates:

$$x_c = \frac{x_{\max} + x_{\min}}{2} \quad y_c = \frac{y_{\max} + y_{\min}}{2} \quad (10)$$

Then, for each GPS point (x_i, y_i) , the radius is computed as:

$$R_i = \sqrt{(x_i - x_c)^2 + (y_i - y_c)^2} \quad (11)$$

and the final cornering radius is obtained by averaging all individual radius estimates:

$$R = \frac{1}{N} \sum_{i=1}^N R_i \quad (12)$$

To improve robustness, the calculations were repeated for all filtered trial CSV files and the resulting values were averaged, yielding:

$$l = 135 \text{ m} \quad R = 35 \text{ m} \quad (13)$$

Having obtained the track geometry, these parameters can then be implemented in Carmaker, as described in the next section.

To facilitate the execution of the complete data processing pipeline, a `main.py` was created. This script sequentially executes all the steps described in the previous sections.

Having obtained the filtered signals, the `data/` directory is ready for post-processing:

```
data/  
  raw/  
    opendlv/  
    cornering/  
    acc_brake/  
  filtered/  
  metadata.csv
```

As part of the engineering workflow taught in the course, the project pipeline was maintained using Git version control. This allows tracking of changes made during development. The repository can be found [here](#).

4 Carmaker

4.1 Vehicle Parameters

After defining the track geometry, the next step was to create a representative model of the Polestar 3 in Carmaker using the Vehicle Data Set Generator. This tool allows the user to specify known vehicle parameters and automatically generate an initial vehicle model.

The parameters used are summarized in Table 2.

Vehicle Parameter	Value
Vehicle class	Full Size SUV
Unloaded weight	2600 kg
Vehicle length	4899.66 mm
Vehicle width	1965.96 mm
Vehicle height	1614 mm
Wheelbase	2984.5 mm
Front track width	1671.32 mm
Rear track width	1653.54 mm
Turning radius	11.82624 m
Tire size	275 / 55 / R20
Anti-roll bar stiffness (front / rear)	34099 N/m / 15947 N/m
IMU sensor mounting	<code>Sensor.0.pos = [1.9, 0, Z]</code>

Table 2: Vehicle parameters used to generate the initial Carmaker vehicle model.

The majority of these parameters were obtained from publicly available Polestar 3 specifications and then entered into the Carmaker Vehicle Data Set Generator.

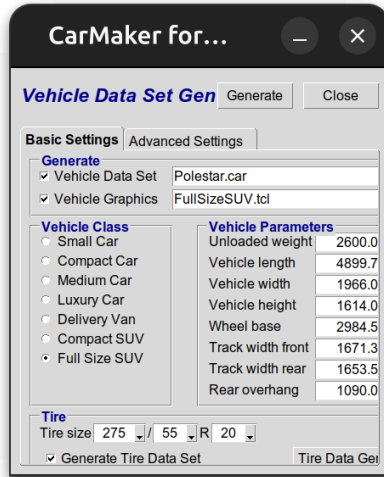


Figure 8: Vehicle Data Set Generator settings.

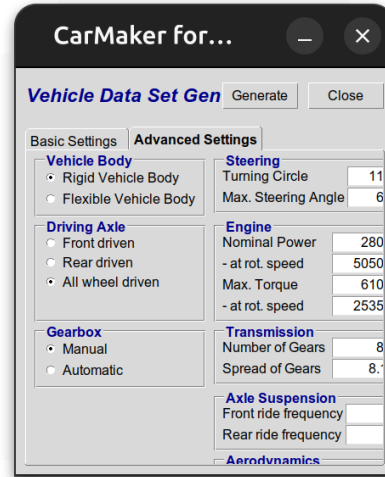


Figure 9: Vehicle Data Set Generator Advanced settings.

Additionally, a virtual IMU was placed in the same location as the physical sensor used during the real test to ensure a meaningful comparison between simulation and test data.

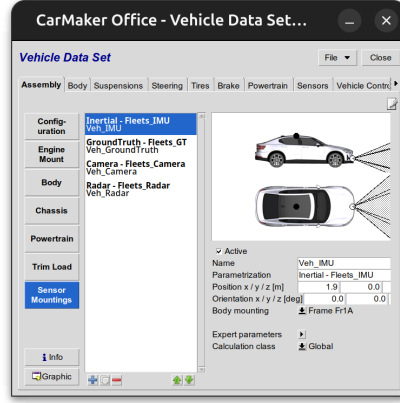


Figure 10: IMU coordinates defined in the Carmaker vehicle model.

Moreover, using the previously identified speed profiles and track geometry, test scenarios were created in the Carmaker Scenario Editor. Figure 11 shows the circular track used for the cornering manoeuvre.

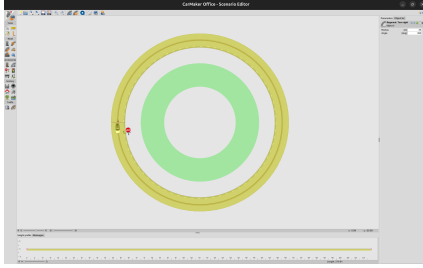


Figure 11: Scenario Editor.

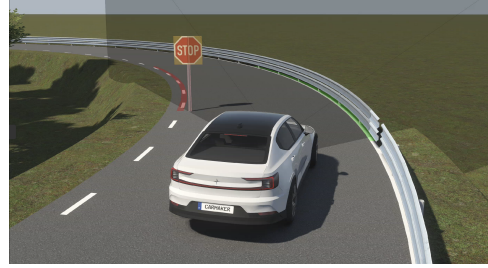


Figure 12: Polestar 3 in IPG Movie.

4.2 Simulation

As part of the objective of evaluating different suspension settings, a parameter sweep was performed for each suspension setting and manoeuvre. Since Polestar 3 uses an air suspension system and the exact suspension parameters of the vehicle were not available, a simplified approach was adopted.

The suspension stiffness adjustments performed during the real tests were approximated by modifying the **amplification** parameters available in Carmaker. These parameters essentially work as a multiplier on both the spring and damper.

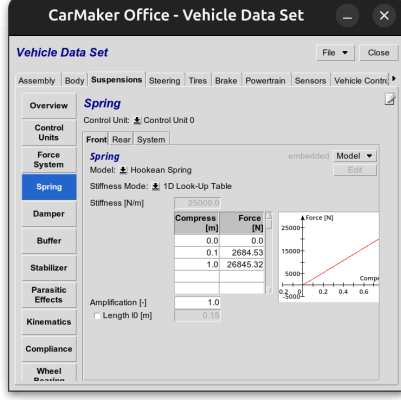


Figure 13: Spring amplification parameter.

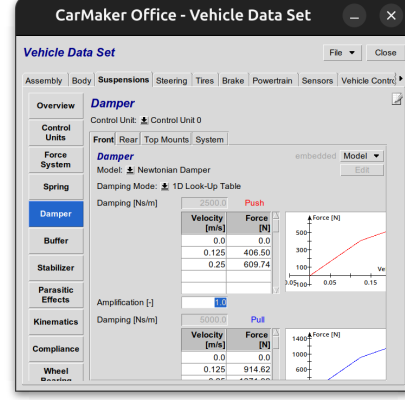


Figure 14: Damper amplification parameter.

The tested amplification values are summarized in Table 3.

Setting	Front	Rear
Default	1.0	1.0
Soft	0.5	0.5
Medium	5.0	5.0
Hard	10.0	10.0

Table 3: Suspension amplification parameters used in the simulation sweep.

After simulating all scenarios, the resulting signals were exported in CSV format and stored in:

data/Carmaker/

directory for subsequent comparison with the filtered data.

5 Results & analysis

In this section, the results from the testing and the model tests are showcased

5.1 Cornering

Looking back at the raw signals shown in Figure 2, it can be observed that the dominant gyroscope signal is the yaw rate ω_z , while ω_x and ω_y mainly consist of small oscillations around zero. This behaviour is expected, as the vehicle primarily rotates about its vertical axis during a constant-radius cornering manoeuvre.

Similarly, lateral acceleration a_y becomes the dominant accelerometer signal due to the centripetal acceleration generated while the vehicle follows the circular path.

Therefore, for the cornering manoeuvre, the most relevant quantities to analyse are:

- Lateral acceleration: a_y
- Yaw rate: ω_z
- Roll rate: ω_x

To facilitate the comparison, the time axes were aligned by subtracting the initial timestamp from each signal using the `iloc` indexing in Python.

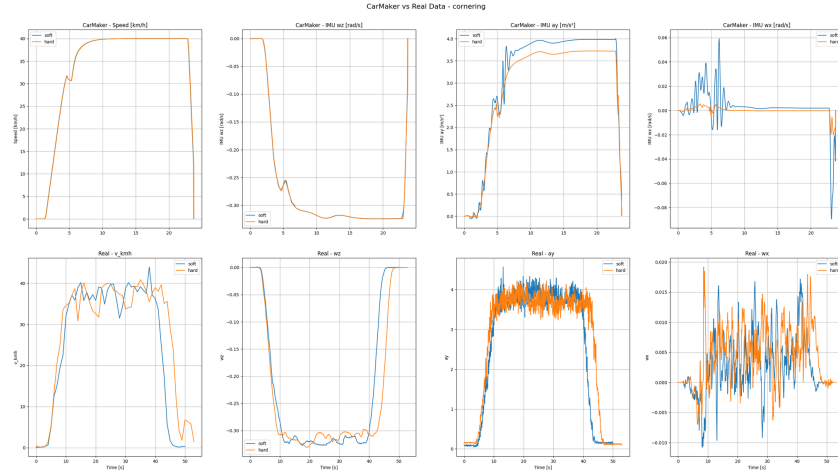


Figure 15: Comparison between Carmaker and experimental measurements during cornering.

5.2 Acceleration & Braking

As for the acceleration and braking manoeuvre, the raw signals show a different behaviour.

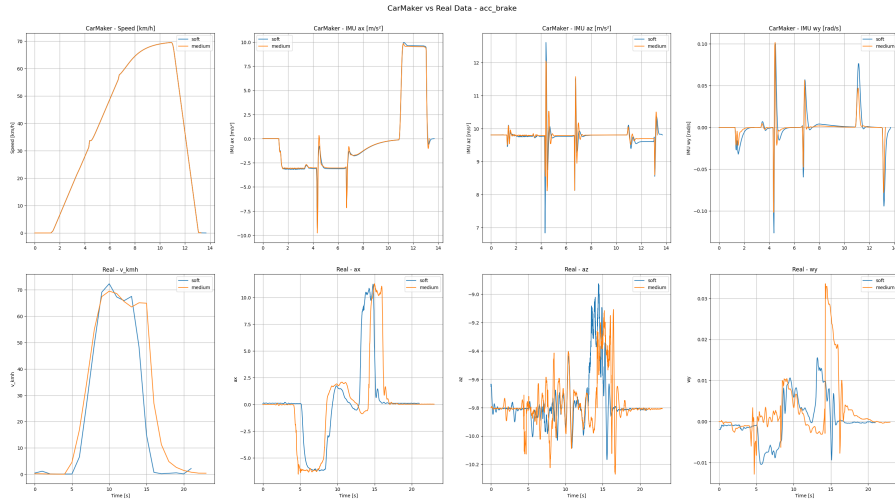


Figure 16: Comparison between Carmaker and experimental measurements during acceleration & braking.

In this case, the most relevant signals are longitudinal acceleration a_x , vertical acceleration a_z , and pitch rate ω_y , as these variables are directly related to longitudinal load transfer and ride comfort during acceleration and braking events.

5.3 Normal Force Estimator

Using sensor data we thought of a Normal force estimator as this is a metric that helps to evaluate ride comfort. The normal loads can be estimated using the linear accelerations and angular velocities. In addition, derivatives of the angular velocities are required, which can be achieved using the `gradient` function.

Therefore we can use the following equations:

$$\begin{aligned}
F_{1z} &= \frac{l_r mg}{2(l_f + l_r)} - \frac{mh}{2(l_f + l_r)} a_x - \frac{m(hC_f(l_f + l_r) - h_{rc}(C_f l_f - C_r l_r))}{w(l_f + l_r)(C_f + C_r)} a_y \\
&\quad + \frac{C_f J_x}{w(C_f + C_r)} \ddot{\varphi}_x - \frac{h_{rc} J_z}{w(l_f + l_r)} \ddot{\varphi}_z + \frac{h_{rc}}{w(l_f + l_r)} M_{z,fx}, \\
F_{2z} &= \frac{l_r mg}{2(l_f + l_r)} - \frac{mh}{2(l_f + l_r)} a_x + \frac{m(hC_f(l_f + l_r) - h_{rc}(C_f l_f - C_r l_r))}{w(l_f + l_r)(C_f + C_r)} a_y \\
&\quad - \frac{C_f J_x}{w(C_f + C_r)} \ddot{\varphi}_x + \frac{h_{rc} J_z}{w(l_f + l_r)} \ddot{\varphi}_z - \frac{h_{rc}}{w(l_f + l_r)} M_{z,fx}, \\
F_{3z} &= \frac{l_f mg}{2(l_f + l_r)} + \frac{mh}{2(l_f + l_r)} a_x - \frac{m(hC_r(l_f + l_r) + h_{rc}(C_f l_f - C_r l_r))}{w(l_f + l_r)(C_f + C_r)} a_y \\
&\quad + \frac{C_r J_x}{w(C_f + C_r)} \ddot{\varphi}_x + \frac{h_{rc} J_z}{w(l_f + l_r)} \ddot{\varphi}_z - \frac{h_{rc}}{w(l_f + l_r)} M_{z,fx}, \\
F_{4z} &= \frac{l_f mg}{2(l_f + l_r)} + \frac{mh}{2(l_f + l_r)} a_x + \frac{m(hC_r(l_f + l_r) + h_{rc}(C_f l_f - C_r l_r))}{w(l_f + l_r)(C_f + C_r)} a_y \\
&\quad - \frac{C_r J_x}{w(C_f + C_r)} \ddot{\varphi}_x - \frac{h_{rc} J_z}{w(l_f + l_r)} \ddot{\varphi}_z + \frac{h_{rc}}{w(l_f + l_r)} M_{z,fx}.
\end{aligned}$$

Figure 17: Normal force estimation equations.

Looking at the equations there's still parameters that are unknown, namely C_f , C_r , and h_{rc} , corresponding to the roll stiffness and roll center height. These were assumed to be constant as for the remaining parameters they're obtained using the Carmaker vehicle dataset.

From the model dataset, the following values were obtained for each axle:

Roll center height Front [m]	Roll center height Rear [m]
0.209	0.090

Table 4: Roll center heights obtained from the vehicle dataset.

To simplify the model, the overall roll center height was approximated as

$$h_{rc} = \frac{l_r}{L} h_{rc,front} + \frac{l_f}{L} h_{rc,rear} \quad (14)$$

The axle roll stiffness was estimated as

$$c_{axle,roll} = k_s \frac{w^2}{2} + k_{arb} \quad (15)$$

Where k_s and k_{arb} represent the spring and anti-roll bar stiffness, respectively. In this work, only k_s was varied between suspension settings.

Figure 18 shows the sum of left and right side estimated loads for each setting. Notice that once the vehicle turns left, the right side load, represented by orange and red curves, increases while the left one decreases which is expected due to lateral load transfer.

Besides both methods being able to capture the same behaviour, the differences between the suspension settings are very small, especially for the real data. For the simulated data, although the differences are still limited, it can be observed that the medium setting generates slightly less load transfer than the soft setting, which is consistent with the expected effect of increased stiffness. Even though the effect is relatively small, this observation provides confidence that

the model captures the expected physical trends and can be used as a starting point for further improvements in model fidelity.

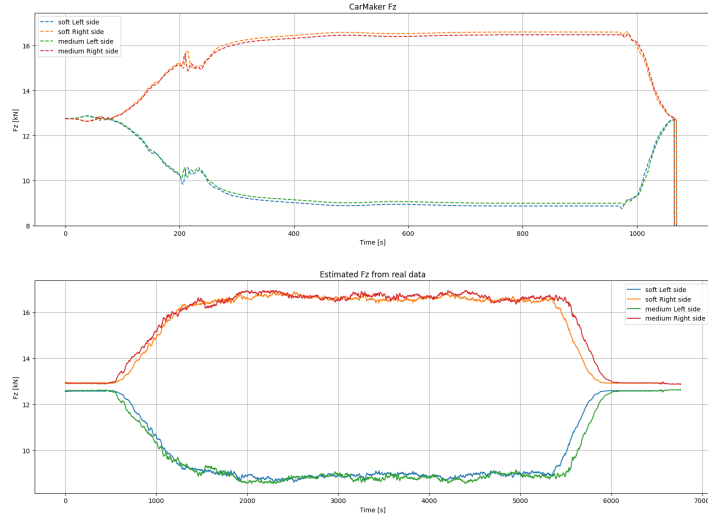


Figure 18: Comparison between Carmaker ground-truth and estimated F_z during cornering.

5.4 Sensitivity Maps

As briefly mentioned in the previous section if we solely based our design decisions on experimental measurements, it is difficult to determine which suspension setting provides the best ride comfort or next best design. However, after comparing the experimental results with the Carmaker simulations, it can be observed that the digital twin reproduces the overall trends reasonably well.

This makes it possible to use the simulation model to generate sensitivity maps and investigate how different suspension settings influence ride comfort metrics.

For the cornering manoeuvre, sensitivity maps were generated for the roll rate ω_x and lateral acceleration a_y .

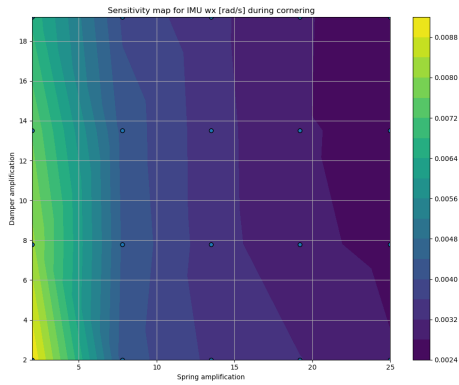


Figure 19: 95th percentile of $|\omega_x|$.

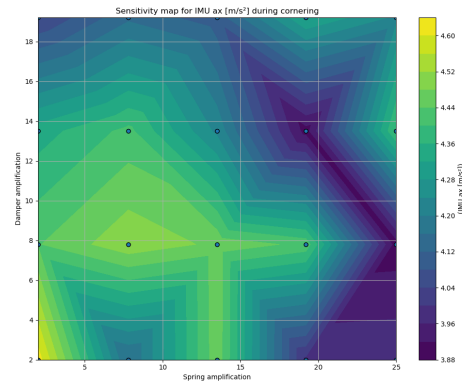


Figure 20: 95th percentile of $|a_y|$.

Similarly, for the acceleration and braking manoeuvre, sensitivity maps were generated for the pitch rate ω_y and vertical acceleration a_z .

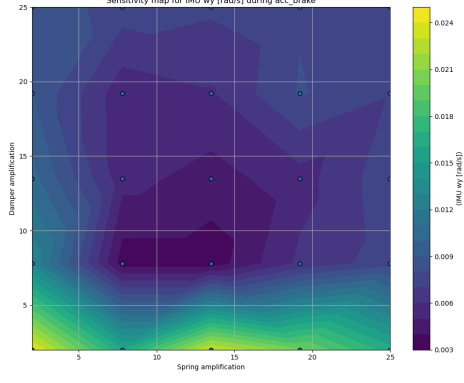


Figure 21: 95th percentile of $|\omega_y|$.

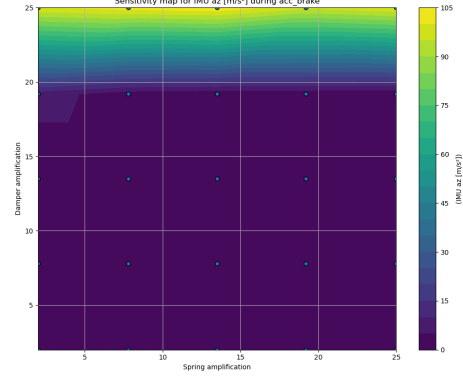


Figure 22: 95th percentile of $|a_z|$.

5.5 Cost function

To combine the sensitivity maps into a single tuning metric, a normalized weighted cost function J was introduced. The objective of this cost function is to identify the suspension setting that provides the best compromise between ride comfort and vehicle response. The cost function was defined based on three main parameters from the simulation:

- 95th percentile of a_z for ride comfort and ω_x for roll and body motion
- Maximum lateral load transfer using ground-truth F_z

Since each metric has different physical units and magnitudes, all quantities were normalized between 0 and 1 which prevents one metric from dominating the optimization.

$$J = 0.50 a_{z,\text{norm}} + 0.25 \omega_{x,\text{norm}} + 0.25 LT_{\text{norm}} \quad (16)$$

From this we choose to put a higher weight into a_z since ride comfort is considered to be the primary goal for this task. Therefore, we can sweep over a range of amplification settings and find which combination optimizes our system, this again, only using the model.

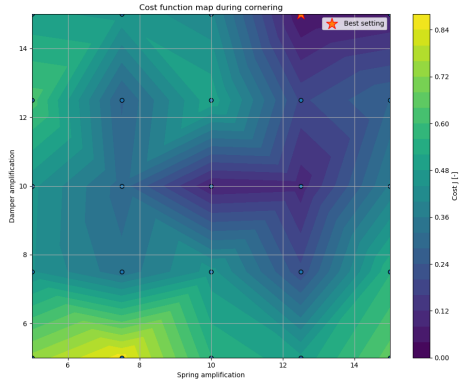


Figure 23: Cost function for cornering data.

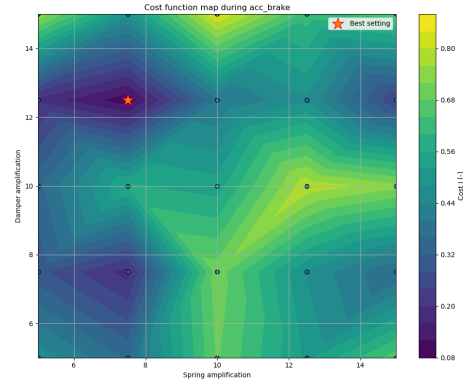


Figure 24: Cost function for acceleration & brake.

From the cost function maps it can be seen that both maneuvers require different optimal suspension settings, which is expected since cornering and acceleration/braking will show different vehicle behaviors.

As seen above, cornering tends to favor stiffer spring configurations with medium damping, while acceleration and braking favor softer springs with slightly higher damping to improve comfort.

Therefore, no single passive setup is optimal for all conditions. This motivates the use of an adaptive suspension system operating around an intermediate region, approximately between spring amplification values of 7.5–10 and damping amplification values of 12–14 depending on the maneuver.

6 Discussion

6.1 Limitations of the Test Cases and scalability

Since the tests were conducted on a controlled test track, the gathered data does not reflect the naturalistic driving conditions typical of a large fleet of vehicles. Performing the experiments in this controlled environment was necessary to simplify the initial parameterization and generation of the CarMaker model. While utilizing real-world fleet data would be possible, it would demand a significantly more sensitive IMU and highly detailed knowledge of the vehicle's inherent suspension parameters.

When developing a suspension system both normal driving and extreme driving scenarios need to be assessed. In order to make this project feasible, we decided to only assess the extreme cases. For a complete suspension setup both data sets are equally vital where the normal data set would be gathered from every day driving by drivers all around the world.

The system at its core does not require any complex sensors and is therefore well suited to being implemented on existing fleets of vehicles. With its main limiting factor in scalability on personal vehicles being the breach of privacy that the GNSS tracking could cause. This would however not be an issue for corporate vehicles where tracking would be more acceptable.

6.2 Future work

Another improvement would be the addition of sensors mounted directly on the suspension, such as potentiometers, to measure the spring displacement at each wheel. By having access to these measurements, it would be possible to obtain a more accurate suspension behaviour, which could improve the accuracy of the vehicle model.



Figure 25: Linear Potentiometer

This could be complemented by introducing a new testing scenario consisting of driving the vehicle over a bump. This test would better excite the suspension dynamics and provide information regarding the damping of the system such as the oscillation frequencies and settling time. This could be further compared with the simulation results.

In addition to our pipeline, we could also focus on damping analysis through damper velocity histograms. These histograms show the damper operating regions and its transient behaviors associated with load transfer events. Essentially, this would help to understand the rate at which load transfers occur, which in consequence has a direct effect on ride comfort. In general, smoother and more gradual load transfers tend to improve ride comfort, whereas rapid load transfers are often perceived as less comfortable by the occupants.